

FASTR Analytics Platform – Tech Stack Documentation

The FASTR Analytics Platform is a health data analytics application that supports multi-tenant project management, executable data modules, and AI-assisted visualisation and slide deck generation. It is deployed on DigitalOcean infrastructure, with server instances provisioned and managed through a purpose-built command-line tool. This document summarises the platform's architecture across the frontend, backend, database, deployment, and operational tooling.

Frontend

The Frontend is built with **SolidJS** in strict Typescript, bundled with **Vite** and **TailwindCSS** for styling and **@solidjs/router** for client-side navigation.

Clerk, a software as a service, is used for authentication on this site and is managed through the **Clerk JS library**.

The platform relies heavily on **panther**, an in-house **visualisation** and rendering library that provides the core **UI** components, chart types (time series, sankey, maps, tables, vertical and horizontal charts), and the underlying rendering system used across the application. Panther is designed as a multi-target renderer, so the same figure definition can be drawn to an on-screen Canvas, embedded into a generated PDF, or exported into a PPTX slide deck. It also provides shared primitives for layout, typography, colour, markdown, and figure styling, giving the platform a consistent visual language regardless of output format.

Various external libraries are used throughout the project:

- The panther lib is used for **UI components** and for **visualisation rendering** on Canvas, PDF, PPTX.
- **File uploads** use Uppy with the TUS resumable upload protocol.
- **Document export** is handled by jsPDF, pptxgenjs, and docx.
- PapaParse for **CSV processing**.
- Zod is used to **validate definition files** that are used for the data module execution.
- The **AI features** uses the anthropic-ai/sdk.

Backend

The backend is a REST/JSON service running on **Deno v2.5.3**, built on the **Hono web framework**. Authentication is aligned with the frontend through **Clerk**, integrated via the hono/clerk-auth and @clerk/backend middleware, ensuring a single source of truth for identity across the stack.

Database access is implemented directly with **postgres.js** using parameterised SQL, without an ORM layer. This design choice preserves full control over query behaviour and avoids abstraction overhead in analytics workloads.

Long-running operations — most notably R-based data module executions — are dispatched to **Deno Web Workers**, with progress streamed to the client in real time via Server-Sent Events (SSE). Caching is delegated to **Valkey**, which runs as a separate container alongside the application.

Database

The platform uses PostgreSQL with a two-tier multi-tenant architecture:

- A **central main database** holds users, instance configuration, projects, and shared reference data.
- Each project gets its own **isolated project_{uuid} database** containing visualisations, reports, module results, and AI interpretations.

Schema migrations are handled by a Deno-based auto-running migration system.

External Integrations

The platform integrates with three external services:

- **DHIS2**, for importing routine health data from partner systems.
- **The Anthropic Claude API**, which powers AI-assisted visualisation generation and slide deck authoring.
- **Clerk**, providing authentication and user management as a managed SaaS identity provider.

Docker and Deployment

The platform is deployed on a **Digital Ocean Droplet** using **Docker containers**. Because the server is packaged in a Docker container, it would be relatively straightforward to deploy a country instance on different infrastructure, including on-premise servers if required.

Each country instance runs its own container for the application (based on denoland/deno:ubuntu-2.5.3), **its own container for PostgreSQL**, and **its own container for Valkey**, with data stored on a **Digital Ocean Block Storage volume**. Each instance directory holds its databases, a sandbox for ephemeral R module execution, and uploaded assets.

R data processing modules run via **Docker-in-Docker**, where the app container spawns short-lived R containers.

Incoming traffic is routed through nginx with per-instance SSL certificates managed by certbot. Application images are published to a Docker registry as timroberton/comb:wb-fastr-server-v{version}.

Server CLI

Server lifecycle management is handled by the server-cli project, which compiles to a standalone Deno binary deployed directly on the Digital Ocean Droplet. It provides a **unified interface** for operating the platform at scale, covering three main areas:

- **Instance registry.** A JSON-based registry (servers.json) tracks all instances, with tagging support for bulk operations across groups of servers.
- **Container management.** Wrappers around Docker handle image pulling, container start/stop/restart cycles, and network cleanup.
- **Infrastructure provisioning.** Automated commands configure nginx virtual hosts and trigger certbot SSL provisioning on a per-instance basis.